

Lecture 15: The Bellman Ford Algorithm (SSSP)

Date: August 5, 2026
Instructor: Aniket Basu Roy
Scribe: Shubham Bhilare, Abinaya S

Overview. This lecture studies the single-source shortest path (SSSP) problem in a directed, edge-weighted graph. Dijkstra’s algorithm gives the shortest-path distances when all edge weights are non-negative, while Bellman–Ford extends the approach to graphs with negative-weight edges and detects negative-weight cycles reachable from the source.

1 Problem Formalization

Definition 1. Consider a directed, edge-weighted graph

$$G = (V, E, w),$$

a source vertex $s \in V$, and an edge-weight function

$$w : E \rightarrow \mathbb{R}.$$

Shortest-path distance. For a vertex v , let $\delta(s, v)$ be the minimum total weight of any directed path from s to v . If v is unreachable from s , then

$$\delta(s, v) = +\infty.$$

A shortest-path tree stores, for each reachable v , a predecessor $\pi[v]$ on a shortest path from s to v .

Output. Compute shortest paths from s to every $v \in V$, with an assumption that there are no negative cycles in G .

Negative cycle. A directed cycle C is negative when

$$\sum_{e \in C} w(e) < 0.$$

If a negative cycle is reachable from s , traversing it repeatedly makes the walk length arbitrarily small, so a finite shortest-path distance does not exist for vertices reachable after the cycle.

Relaxation. For an edge (u, v) , the relaxation operation checks whether a newly discovered route through u is better than the current estimate of v . The ‘better’ route is decided as to be the minimum weighted value for each vertex from s to v . This is defined as:

$$\begin{aligned} \text{Relax}(u, v) : \\ \text{if } d[u] + w(u, v) < d[v] : \\ \quad d[v] \leftarrow d[u] + w(u, v) \\ \quad \pi[v] \leftarrow u \end{aligned}$$

Note. After $|V| - 1$ rounds, if there is an edge (u, v) with finite $d[u]$ such that

$$d[u] + w(u, v) < d[v],$$

then a reachable negative-weight cycle exists.

2 Dynamic Programming approach

2.1 Dynamic-programming state

Definition 2. For $i \geq 0$ and $v \in V$, define $\text{OPT}(i, v)$ as the minimum length of any walk from s to v using at most i edges.

Base state.

$$\text{OPT}(0, s) = 0, \quad \text{OPT}(0, v) = +\infty \quad \text{for every } v \neq s.$$

Final state. Every shortest path can be chosen uses at most $|V| - 1$ edges,

$$\text{OPT}(|V| - 1, v) = \delta(s, v) \quad \text{for every } v.$$

Vertex / edge budget	0	1	$\dots$	$ V - 1$
v_1	$\text{OPT}(i, v_1)$	$\text{OPT}(i, v_1)$	$\dots$	$\text{OPT}(i, v_1)$
v_2	$\text{OPT}(i, v_2)$	$\text{OPT}(i, v_2)$	$\dots$	$\text{OPT}(i, v_2)$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
v_n	$\text{OPT}(i, v_n)$	$\text{OPT}(i, v_n)$	$\dots$	$\text{OPT}(i, v_n)$

2.2 Recurrence and negative cycles

A shortest bounded walk either uses at most $i - 1$ edges, or it uses exactly i edges and ends with some edge (u, v) . Therefore,

$$\text{OPT}(i, v) = \min \left\{ \text{OPT}(i - 1, v), \min_{(u,v) \in E} [\text{OPT}(i - 1, u) + w(u, v)] \right\}, \quad i \geq 1. \quad (1)$$

Interpretation. The first term means that allowing one more edge does not force us to use it. The second term considers every possible predecessor u of v and appends the edge (u, v) to the best walk to u using at most $i - 1$ edges.

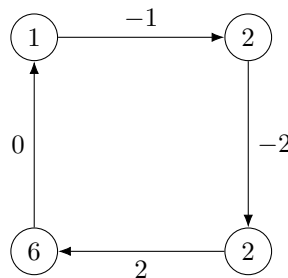


Figure 1: Illustration of a directed cycle; a cycle is negative precisely when the sum of its edge weights is strictly negative.

3 Proof of Correctness

Theorem 1 (Correctness of the DP recurrence). *For every $i \geq 0$ and every vertex v , $\text{OPT}(i, v)$ equals the length of the shortest walk from s to v using at most i edges.*

Proof. We use induction on i .

For $i = 0$, the only walk with at most zero edges is the empty walk at s , giving

$$\text{OPT}(0, s) = 0, \quad \text{OPT}(0, v) = +\infty \quad \text{for } v \neq s.$$

Assume the statement holds for $i - 1$. Consider a shortest walk counted by $\text{OPT}(i, v)$.

If it uses at most $i - 1$ edges, its value is

$$\text{OPT}(i - 1, v).$$

Otherwise, its final edge is (u, v) , and the prefix from s to u uses at most $i - 1$ edges. By the induction hypothesis, that prefix has minimum possible length

$$\text{OPT}(i - 1, u).$$

Therefore the walk has length

$$\text{OPT}(i-1, u) + w(u, v),$$

and minimizing over all incoming edges to v gives the recurrence. So, $\text{OPT}(i, v)$ is correct. $\square$

Theorem 2 (Bellman–Ford correctness without reachable negative cycles). *If no negative-weight cycle is reachable from s , then after $|V| - 1$ relaxation rounds the distance $d[v]$ equals $\delta(s, v)$ for every v .*

Proof. Every shortest path can be chosen simple, because if a shortest walk repeated a vertex then it would contain a cycle. With no reachable negative cycle, removing any non-negative repeated cycle cannot increase the path length.

Hence some shortest path to v uses at most $|V| - 1$ edges. The bounded-walk dynamic program therefore gives

$$\text{OPT}(|V| - 1, v) = \delta(s, v).$$

Bellman–Ford implements the same relaxations without storing the entire table, so its final estimates are correct. $\square$

Theorem 3 (Negative-cycle test). *If a reachable negative-weight cycle exists, Bellman–Ford can still improve some reachable distance after $|V| - 1$ full passes. Conversely, if such an improvement is possible after $|V| - 1$ passes, a reachable negative-weight cycle must exist.*

Proof. A walk with at least $|V|$ edges repeats a vertex. If the repeated-vertex portion had non-negative weight, deleting that cycle would produce a walk with fewer edges and no larger weight, contradicting the strict improvement over all walks represented using at most $|V| - 1$ edges.

Thus the repeated cycle must be negative. $\square$

4 Algorithms and Execution Trace

4.1 Dijkstra algorithm

Correctness invariant. Whenever a vertex u is removed from the priority queue, $d[u]$ is the true shortest-path distance $\delta(s, u)$. The non-negative edge-weight condition is what makes the greedy finalization safe.

4.2 Bellman–Ford algorithm

Algorithm 1 Bellman–Ford(G, w, s)

```

1:  $d[s] \leftarrow 0$ 
2: for all  $v \in V, v \neq s$  do
3:    $d[v] \leftarrow +\infty$ 
4: for all  $v \in V$  do
5:    $\pi[v] \leftarrow \text{NIL}$ 
6: for  $k \leftarrow 1$  to  $|V| - 1$  do
7:   for all edges  $(u, v) \in E$  do
8:     if  $d[u] \neq +\infty$  and  $d[v] > d[u] + w(u, v)$  then
9:        $d[v] \leftarrow d[u] + w(u, v)$ 
10:       $\pi[v] \leftarrow u$ 
11: for all edges  $(u, v) \in E$  do
12:   if  $d[u] \neq +\infty$  and  $d[v] > d[u] + w(u, v)$  then
13:     report a reachable negative-weight cycle
14: return  $d, \pi$ 
```

DP implementation. The explicit DP table computes $\text{OPT}(i, v)$ for every pair (i, v) . The standard in-place Bellman–Ford implementation keeps only the current best distance per vertex, reducing auxiliary space while retaining the $O(|V||E|)$ running time.

4.3 Concrete execution trace

Consider vertices

$$\{s, a, b, c, d\}$$

with directed edges

$$s \rightarrow a (4), \quad s \rightarrow b (5), \quad a \rightarrow b (-2), \quad a \rightarrow c (4), \quad b \rightarrow c (3), \quad c \rightarrow d (-1).$$

There is a negative edge but no negative cycle.

Step-by-Step Iterations Pass 1

Relaxing all edges in order:

Edge (s, a) with weight 4 :	$d(a) = \min(\infty, 0 + 4) = 4$	$\pi(a) = s$
Edge (s, b) with weight 5 :	$d(b) = \min(\infty, 0 + 5) = 5$	$\pi(b) = s$
Edge (a, b) with weight -2 :	$d(b) = \min(5, 4 + (-2)) = 2$	$\pi(b) = a$
Edge (b, c) with weight 3 :	$d(c) = \min(\infty, 2 + 3) = 5$	$\pi(c) = b$
Edge (c, d) with weight -1 :	$d(d) = \min(\infty, 5 + (-1)) = 4$	$\pi(d) = c$

After Pass 1:

Distances: $d(s) = 0$, $d(a) = 4$, $d(b) = 2$, $d(c) = 5$, $d(d) = 4$

Pass 2

Relaxing all edges again:

Edge (s, a) :	$d(a) = \min(4, 0 + 4) = 4$	(No change)
Edge (s, b) :	$d(b) = \min(2, 0 + 5) = 2$	(No change)
Edge (a, b) :	$d(b) = \min(2, 4 + (-2)) = 2$	(No change)
Edge (b, c) :	$d(c) = \min(5, 2 + 3) = 5$	(No change)
Edge (c, d) :	$d(d) = \min(4, 5 + (-1)) = 4$	(No change)

Since no distances changed during Pass 2, the distances have fully converged. Passes 3 and 4 will yield no further changes. Therefore the final distances are

$$d(s) = 0, \quad d(a) = 4, \quad d(b) = 2, \quad d(c) = 5, \quad d(d) = 4.$$

i	$\text{OPT}(i, s)$	$\text{OPT}(i, a)$	$\text{OPT}(i, b)$	$\text{OPT}(i, c)$	$\text{OPT}(i, d)$
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	4	5	$+\infty$	$+\infty$
2	0	4	2	8	$+\infty$
3	0	4	2	5	7
4	0	4	2	5	4

Table 1: Dynamic-programming trace for the sample graph.

5 Reconstructing the Shortest Paths

Predecessor array. Whenever RELAX improves $d[v]$ through u , store

$$\pi[v] = u.$$

Once the distances are final, repeatedly follow predecessors from a destination v back toward s . Reversing the obtained sequence gives the shortest path from s to v .

Example reconstruction. For the execution trace above,

$$\pi[a] = s, \quad \pi[b] = a, \quad \pi[c] = b, \quad \pi[d] = c.$$

Thus the shortest path to d is recovered as

$$d \leftarrow c \leftarrow b \leftarrow a \leftarrow s,$$

which reverses to

$$s \rightarrow a \rightarrow b \rightarrow c \rightarrow d.$$

Unreachable vertices. If $d[v]$ remains $+\infty$, then v is unreachable from s and no shortest path from s to v exists. Its predecessor remains NIL.

Negative-cycle case. When a reachable negative cycle is detected, predecessor pointers cannot in general be interpreted as a finite shortest-path tree for vertices whose values can decrease without bound. The algorithm should report the existence of the cycle rather than claiming a finite optimum for affected vertices.

6 Complexity Analysis of DP

6.1 Running Time and Space Complexity Analysis

Algorithm / form	Assumption / implementation	Time	Auxiliary space
Dijkstra	Binary min-heap	$O((V + E) \log V)$	$O(V)$
Dijkstra	Array / dense implementation	$O(V ^2)$	$O(V)$
Bellman–Ford	In-place relaxation	$O(V E)$	$O(V)$

Table 2: Standard complexity bounds.

DP-table space. If every $\text{OPT}(i, v)$ value is stored explicitly, there are $|V|$ choices of vertex and $|V|$ edge-budget states, giving space,

$$O(|V|^2)$$

The standard Bellman–Ford implementation is an optimization that keeps only the current distance and predecessor arrays, giving auxiliary space,

$$O(|V|)$$

Sparse graphs. When $|E| = O(|V|)$ Bellman–Ford becomes $O(|V|^2)$ in the worst case.

Complete graphs. When $|E| = \frac{n(n-1)}{2}$ so $O(\frac{n(n-1)}{2} * (n-1)) = O(n^3)$ in the worst case.

Early termination. Bellman–Ford may stop before $|V| - 1$ passes if a full pass performs no relaxation: no distance can improve in later passes. (As seen in the tracing example above.)